LAB

10

Operating Systems · FCIM / FAF · UTM · 2026-2027

Add Your Own System Call: trace

B **Track B: build your own small OS (xv6) · 90 minutes**

AT A GLANCE

Part 1	The number and the user side	~25 min
Part 2	The kernel side	~35 min
Part 3	Play with the mask	~25 min

Goal

For the first time you change the kernel itself. You add a new system call to xv6, trace, that makes a process print every system call it makes. On Linux, strace did this for you in Lab 9; today YOU build it. You follow one number, 23, from a user program down to a function in the kernel.

At the end this works:

TERMINAL

```
$ trace 32 grep hello README
3: syscall read -> 1023
3: syscall read -> 965
3: syscall read -> 453
3: syscall read -> 0
```

Before you start

- Use your ~/xv6-riscv tree. Check that it builds and boots: make qemu, then Ctrl-A X to leave.
- Every block of code below is complete. Type it (or copy it) exactly where the sheet says. After each part there is a check: do not go on until it passes.

Careful: our xv6 is newer than the 6.1810 page on the internet

The MIT instructions say SYS_trace is 22 and the kernel prints with printf. In our tree 22 is already sync, and the kernel print function is called printk. Use 23 and printk, as below.

Minimum to pass

Parts 1 and 2 done and trace 32 grep hello README printing the read lines. Part 3 brings the rest of the grade.

PART 1 The number and the user side ~25 min

First the call gets a number, then the user side learns to ask for it. Five small edits.

1.1 kernel/syscall.h: the number

Add one line at the end of the file:

FILE kernel/syscall.h

```
#define SYS_sync   22
#define SYS_trace  23      // new
```

1.2 user/user.h: the declaration

Next to the other system calls (after int sync(void);):

FILE user/user.h

```
int sync(void);
int trace(int);          // new
```

1.3 user/usys.pl: the stub that runs ecall

Add one line at the end. At build time this script writes user/usys.S, where trace becomes three instructions: li a7, SYS_trace; ecall; ret.

FILE user/usys.pl

```
entry("sync");
entry("trace");          # new
```

1.4 user/trace.c: the program

Create the file with exactly this content. It sets the mask on itself, then becomes the command with exec (the mask stays, because exec keeps the process):

FILE user/trace.c

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if (argc < 3) {
    fprintf(2, "usage: trace mask command [args...]\n");
    exit(1);
  }
  if (trace(atoi(argv[1])) < 0) {
    fprintf(2, "trace: failed\n");
    exit(1);
  }
  exec(argv[2], argv + 2);
  fprintf(2, "trace: exec %s failed\n", argv[2]);
  exit(1);
}
```

1.5 Makefile: put trace on the xv6 disk

In the UPROGS list, after $U/_zombie\, add one line. It starts with a TAB, and nothing may follow the backslash:

FILE Makefile

```
	$U/_zombie\
	$U/_trace\
```

Check 1: the number reaches the kernel

make qemu, then run trace. The kernel has no function for 23 yet, so it must complain, exactly like this:

TERMINAL

```
$ trace 32 grep hello README
3 trace: unknown sys call 23
trace: failed
```

This message is good news: your program ran ecall with a7 = 23 and the kernel received it. If instead the build fails, read the first error: usually a missing TAB in the Makefile or a typo in usys.pl.

PART 2 The kernel side ~35 min

2.1 kernel/proc.h: a place for the mask

Each process remembers its own mask. In struct proc, after the name field:

FILE kernel/proc.h

```
  char name[16];               // Process name (debugging)
  int trace_mask;              // new: which calls to print
```

2.2 kernel/sysproc.c: the handler

Add this function at the end of the file. It reads argument 0 (register a0) and stores it on the process:

FILE kernel/sysproc.c

```
uint64
sys_trace(void)
{
  int mask;
  argint(0, &mask);
  myproc()->trace_mask = mask;
  return 0;
}
```

2.3 kernel/syscall.c: the table

Two lines: tell the file the function exists, and give it place 23 in the table.

FILE kernel/syscall.c

```
extern uint64 sys_sync(void);
extern uint64 sys_trace(void);   // new
...
  [SYS_sync]    = sys_sync,
  [SYS_trace]   = sys_trace,       // new
```

2.4 kernel/syscall.c: the names and the print

The kernel only knows numbers. Add a table of names just ABOVE the function syscall():

FILE kernel/syscall.c

```
static char *syscallnames[] = {
  [SYS_fork] "fork",   [SYS_exit] "exit",     [SYS_wait] "wait",
  [SYS_pipe] "pipe",   [SYS_read] "read",     [SYS_kill] "kill",
  [SYS_exec] "exec",   [SYS_fstat] "fstat",   [SYS_chdir] "chdir",
  [SYS_dup] "dup",     [SYS_getpid] "getpid", [SYS_sbrk] "sbrk",
  [SYS_pause] "pause", [SYS_uptime] "uptime", [SYS_open] "open",
  [SYS_write] "write", [SYS_mknod] "mknod",   [SYS_unlink] "unlink",
  [SYS_link] "link",   [SYS_mkdir] "mkdir",   [SYS_close] "close",
  [SYS_sync] "sync",   [SYS_trace] "trace",
};
```

Then, inside syscall(), right after the line that runs the call, add the two lines marked new. After the call, so the return value in a0 is already there:

FILE kernel/syscall.c

```
    p->trapframe->a0 = syscalls[num]();
    if ((p->trace_mask >> num) & 1)                    // new
      printk("%d: syscall %s -> %ld\n",                // new
             p->pid, syscallnames[num], p->trapframe->a0);
```

2.5 kernel/proc.c: children inherit the mask

In fork(), right after the line that copies the name:

FILE kernel/proc.c

```
  safestrcpy(np->name, p->name, sizeof(p->name));
  np->trace_mask = p->trace_mask;      // new
```

Check 2: trace works

Ctrl-A X, make qemu, then:

TERMINAL

```
$ trace 32 grep hello README
3: syscall read -> 1023
3: syscall read -> 965
3: syscall read -> 453
3: syscall read -> 0
```

Your numbers must be exactly these: 1023 + 965 + 453 = 2441, the size of README, read in three pieces, then 0 for the end of the file. The pid can differ.

If it does not work

Still "unknown sys call 23": the table entry in 2.3 is missing. Nothing is printed: the new lines in 2.4 are missing or placed before the call. Build error about printf: write printk. Build error about syscallnames: the table must be ABOVE syscall().

PART 3 Play with the mask ~25 min

The mask is a number read in binary: bit N on means "print call number N". 32 = 2^5, and 5 is SYS_read. Run the commands below and answer in notes.txt.

3.1 All the calls (all bits on):

TERMINAL

```
$ trace 2147483647 grep hello README
4: syscall trace -> 0
4: syscall exec -> 3
4: syscall open -> 3
4: syscall read -> 1023
4: syscall read -> 965
4: syscall read -> 453
4: syscall read -> 0
4: syscall close -> 0
```

3.2 Only fork (bit 1, mask 2), on a test that forks until the process table is full:

TERMINAL

```
$ trace 2 usertests forkforkfork
usertests starting
test forkforkfork: 6: syscall fork -> 7
7: syscall fork -> 8
8: syscall fork -> 9
...
8: syscall fork -> 67
8: syscall fork -> -1
OK
ALL TESTS PASSED
```

3.3 Only write (bit 16, mask 65536):

TERMINAL

```
$ trace 65536 echo hi
hi69: syscall write -> 2

69: syscall write -> 1
```

Observe (write in notes.txt)

1. In `trace 32 grep hello README` why do you see only read, and not open or close?
2. In forkforkfork, three different pids print lines. Which line of code from Part 2 makes that possible?
3. In `trace 65536 echo hi` the trace line is printed between "hi" and the new line. Why? (How many times does echo call write?)
4. Which mask traces both open
15. and close (21), and nothing else?

Optional (not graded)

Add a second system call, sysinfo, that returns the number of free memory bytes and the number of processes in use. Same recipe (number 24, stub, table, handler), plus one helper in kernel/kalloc.c and one in kernel/proc.c.

What you hand in

- Your modified xv6 tree (committed to git), with trace working.